

# Interview Notification Toolkit

Full source code listing

|                                                     |                                  |
|-----------------------------------------------------|----------------------------------|
| <a href="#">task5.py</a>                            | Flask email notification service |
| <a href="#">templates/email_template.html</a>       | Email HTML template              |
| <a href="#">Interview_Lifecycle_Tracker/main.py</a> | Winter interview tracker GUI     |
| <a href="#">tests/test_task5.py</a>                 | Pytest suite for task5.py        |
| <a href="#">requirements.txt</a>                    | Python dependencies              |
| <a href="#">.env.example</a>                        | Environment variable template    |
| <a href="#">.gitignore</a>                          | Git ignore rules                 |
| <a href="#">README.md</a>                           | Project documentation            |

## task5.py

*Flask email notification service*

```
"""
task5.py - Email notification service.

Sends interview-status emails via SendGrid. Exposes a single POST endpoint,
/send_email, protected by a shared-secret API key and a simple in-memory
rate limiter, plus a GET / health-check route.

See README.md for setup, environment variables, and the documented trust
boundary for this service.
"""

import os
import re
import logging
from collections import defaultdict, deque
from functools import wraps
from time import time

from dotenv import load_dotenv
from flask import Flask, jsonify, render_template, request
from sendgrid import SendGridAPIClient
from sendgrid.helpers.mail import Mail

load_dotenv()

app = Flask(__name__)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("task5")

# --- Configuration -----

SENDGRID_API_KEY = os.environ.get("SENDGRID_API_KEY")
FROM_EMAIL = os.environ.get("FROM_EMAIL", "notifications@example.com")
API_KEY = os.environ.get("APP_API_KEY") # shared-secret required on /send_email

RATE_LIMIT_WINDOW_SECONDS = int(os.environ.get("RATE_LIMIT_WINDOW_SECONDS", "60"))
RATE_LIMIT_MAX_REQUESTS = int(os.environ.get("RATE_LIMIT_MAX_REQUESTS", "10"))

EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")

# caller_id -> deque of request timestamps within the current window.
# NOTE: this is per-process, in-memory rate limiting. It's sufficient for a
# single dev/staging instance behind one worker, but will NOT coordinate
# across multiple gunicorn/uwsgi workers or replicas. For a multi-worker
# production deployment, back this with Redis or a similar shared store.
_rate_limit_buckets = defaultdict(deque)

# --- Helpers -----

def is_valid_email(value):
    return bool(value) and bool(EMAIL_RE.match(value.strip()))

def require_api_key(view_func):
    """Require a matching X-API-Key header.

    This is a minimum-viable auth mechanism intended for a trusted internal
    caller (e.g. the Interview_Lifecycle_Tracker GUI or another internal
    service) -- see README.md's "Trust model" section. It is not intended
    to make this endpoint safe to expose directly on the public internet.
    """

    @wraps(view_func)
    def wrapped(*args, **kwargs):
        if not API_KEY:
```

```

        logger.error("APP_API_KEY is not configured; refusing all requests to protected endpoint.")
        return jsonify({"status": "failed", "error": "Server auth is not configured."}), 500
    provided = request.headers.get("X-API-Key")
    if not provided or provided != API_KEY:
        return jsonify({"status": "failed", "error": "Unauthorized."}), 401
    return view_func(*args, **kwargs)

return wrapped

def rate_limited(view_func):
    @wraps(view_func)
    def wrapped(*args, **kwargs):
        caller_id = request.headers.get("X-API-Key") or request.remote_addr or "unknown"
        now = time()
        bucket = _rate_limit_buckets[caller_id]
        while bucket and now - bucket[0] > RATE_LIMIT_WINDOW_SECONDS:
            bucket.popleft()
        if len(bucket) >= RATE_LIMIT_MAX_REQUESTS:
            return jsonify({"status": "failed", "error": "Rate limit exceeded. Try again later."}), 429
        bucket.append(now)
        return view_func(*args, **kwargs)

    return wrapped

# --- Routes -----

@app.route("/", methods=["GET"])
def health_check():
    return jsonify({"status": "ok"}), 200

@app.route("/send_email", methods=["POST"])
@require_api_key
@rate_limited
def send_email():
    data = request.get_json(silent=True) or {}
    recipient = (data.get("recipient") or "").strip()
    candidate = (data.get("candidate") or "").strip()

    if not recipient or not is_valid_email(recipient):
        return jsonify({"status": "failed", "error": "A valid 'recipient' email address is required."}), 400

    if not candidate:
        return jsonify({"status": "failed", "error": "'candidate' is required."}), 400

    if not SENDGRID_API_KEY:
        logger.error("SENDGRID_API_KEY is not configured.")
        return jsonify({"status": "failed", "error": "Email service is not configured."}), 500

    try:
        html_content = render_template("email_template.html", candidate=candidate)
    except Exception:
        logger.exception("Failed to render email template for candidate=%s", candidate)
        return jsonify({"status": "failed", "error": "Failed to prepare email content."}), 500

    message = Mail(
        from_email=FROM_EMAIL,
        to_emails=recipient,
        subject=f"Interview Update for {candidate}",
        html_content=html_content,
    )

    try:
        sg = SendGridAPIClient(SENDGRID_API_KEY)
        response = sg.send(message)
        logger.info(
            "Email sent to %s (candidate=%s) sendgrid_status=%s",
            recipient,
            candidate,

```

```

        getattr(response, "status_code", "unknown"),
    )
    return jsonify({"status": "sent", "recipient": recipient}), 200
except Exception:
    # Log full internal detail server-side; never leak it to the caller.
    logger.exception("SendGrid send failed for recipient=%s candidate=%s", recipient, candidate)
    return (
        jsonify(
            {
                "status": "failed",
                "error": "Failed to send email. Please try again later or contact an administrator.",
            }
        ),
        502,
    )

if __name__ == "__main__":
    debug_mode = os.environ.get("FLASK_DEBUG", "false").strip().lower() in ("1", "true", "yes")
    app.run(debug=debug_mode)

```

## templates/email\_template.html

*Email HTML template*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Interview Update</title>
</head>
<body style="font-family: Arial, sans-serif; color: #222; line-height: 1.5;">
  <p>Hi {{ candidate }},</p>
  <p>
    This is an update regarding your interview process. Your recruiting
    contact will follow up with any additional details.
  </p>
  <p>Thank you,<br>The Hiring Team</p>
</body>
</html>
```

## Interview\_Lifecycle\_Tracker/main.py

*Tkinter interview tracker GUI*

```
"""
Interview_Lifecycle_Tracker/main.py

A small tkinter GUI for logging interview stage changes. Events are written
as structured JSON Lines to a data directory (configurable via
TRACKER_DATA_DIR), with error handling around the file write so a
permissions issue or full disk shows a dialog instead of crashing the app.

Optionally, on relevant stage transitions ("Completed" / "Incomplete"), it
can trigger a notification email via task5.py's /send_email endpoint. This
is off by default and fully non-blocking (the HTTP call runs on a background
thread) so the GUI never freezes while waiting on the network.
"""

import json
import os
import threading
import tkinter as tk
import urllib.error
import urllib.request
from datetime import datetime, timezone
from tkinter import messagebox, ttk

# --- Configuration -----

DATA_DIR = os.environ.get(
    "TRACKER_DATA_DIR",
    os.path.join(os.path.dirname(os.path.abspath(__file__)), "data"),
)
EVENTS_FILE = os.path.join(DATA_DIR, "events.jsonl")

NOTIFY_ENABLED = os.environ.get("TRACKER_NOTIFY_ENABLED", "false").strip().lower() in ("1", "true", "yes")
NOTIFY_URL = os.environ.get("TRACKER_NOTIFY_URL", "http://127.0.0.1:5000/send_email")
NOTIFY_API_KEY = os.environ.get("TRACKER_NOTIFY_API_KEY", "")

NOTIFY_STAGES = {"Completed", "Incomplete"}
STAGES = ["Scheduled", "In Progress", "Completed", "Incomplete", "Cancelled"]

# --- Persistence -----

def ensure_data_dir():
    os.makedirs(DATA_DIR, exist_ok=True)

def log_event(candidate, stage, notes=""):
    """Append a structured event to the JSON Lines log.

    Returns True on success. On failure, shows a messagebox and returns
    False instead of letting the exception propagate out of a GUI callback.
    """
    event = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "candidate": candidate,
        "stage": stage,
        "notes": notes,
    }
    try:
        ensure_data_dir()
        with open(EVENTS_FILE, "a", encoding="utf-8") as f:
            f.write(json.dumps(event) + "\n")
        return True
    except OSError as e:
        messagebox.showerror("Logging Error", f"Could not write event to log file:\n{e}")
        return False

# --- Notification integration -----
def notify_async(candidate, recipient):
```

```
"""Fire-and-forget call to task5.py's /send_email endpoint.
```

```
Runs on a daemon thread so the GUI thread is never blocked on network
I/O. Failures are logged to stdout rather than raised, since this is a
best-effort notification, not the source of truth for tracking.
"""
```

```
def _send():
    try:
        payload = json.dumps({"recipient": recipient, "candidate": candidate}).encode("utf-8")
        req = urllib.request.Request(
            NOTIFY_URL,
            data=payload,
            headers={
                "Content-Type": "application/json",
                "X-API-Key": NOTIFY_API_KEY,
            },
            method="POST",
        )
        with urllib.request.urlopen(req, timeout=10) as resp:
            if resp.status != 200:
                print(f"[notify] Non-200 response ({resp.status}) notifying for {candidate}")
    except (urllib.error.URLError, urllib.error.HTTPError) as e:
        print(f"[notify] Failed to send notification for {candidate}: {e}")

threading.Thread(target=_send, daemon=True).start()
```

```
# --- GUI -----
```

```
class InterviewTrackerApp:
    def __init__(self, root):
        self.root = root
        root.title("Interview Lifecycle Tracker")
        root.geometry("440x300")
        root.resizable(False, False)

        frm = ttk.Frame(root, padding=16)
        frm.pack(fill="both", expand=True)

        ttk.Label(frm, text="Candidate name").pack(anchor="w")
        self.candidate_var = tk.StringVar()
        ttk.Entry(frm, textvariable=self.candidate_var).pack(fill="x", pady=(0, 10))

        ttk.Label(frm, text="Recipient email (for notifications)").pack(anchor="w")
        self.recipient_var = tk.StringVar()
        ttk.Entry(frm, textvariable=self.recipient_var).pack(fill="x", pady=(0, 10))

        ttk.Label(frm, text="Stage").pack(anchor="w")
        self.stage_var = tk.StringVar(value=STAGES[0])
        ttk.Combobox(frm, textvariable=self.stage_var, values=STAGES, state="readonly").pack(fill="x", pady=(0, 10))

        ttk.Label(frm, text="Notes").pack(anchor="w")
        self.notes_var = tk.StringVar()
        ttk.Entry(frm, textvariable=self.notes_var).pack(fill="x", pady=(0, 16))

        ttk.Button(frm, text="Track Interview", command=self.track_interview).pack()

        status_text = "Notifications: enabled" if NOTIFY_ENABLED else "Notifications: disabled"
        status_color = "#1a7f37" if NOTIFY_ENABLED else "#666666"
        ttk.Label(frm, text=status_text, foreground=status_color).pack(pady=(14, 0))
        ttk.Label(frm, text=f"Log file: {EVENTS_FILE}", foreground="#888888").pack(pady=(2, 0))

    def track_interview(self):
        candidate = self.candidate_var.get().strip()
        stage = self.stage_var.get().strip()
        notes = self.notes_var.get().strip()
        recipient = self.recipient_var.get().strip()

        if not candidate:
            messagebox.showwarning("Missing info", "Please enter a candidate name.")
```

```

        return

    if not log_event(candidate, stage, notes):
        return # error already shown to the user

    if NOTIFY_ENABLED and stage in NOTIFY_STAGES:
        if recipient:
            notify_async(candidate, recipient)
        else:
            messagebox.showwarning(
                "Notification skipped",
                "Notifications are enabled but no recipient email was provided.",
            )

    messagebox.showinfo("Tracked", f"Logged '{stage}' for {candidate}.")
    self.notes_var.set("")

def main():
    root = tk.Tk()
    InterviewTrackerApp(root)
    root.mainloop()

if __name__ == "__main__":
    main()

```



## tests/test\_task5.py

*Pytest suite for task5.py*

```
import os
import sys

import pytest

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

# Configure env vars before importing the app module.
os.environ.setdefault("APP_API_KEY", "test-key")
os.environ.setdefault("SENDGRID_API_KEY", "test-sendgrid-key")
os.environ.setdefault("FROM_EMAIL", "test@example.com")

import task5 # noqa: E402

AUTH_HEADERS = {"X-API-Key": "test-key"}

@pytest.fixture
def client():
    task5.app.config["TESTING"] = True
    with task5.app.test_client() as c:
        yield c

@pytest.fixture(autouse=True)
def _reset_rate_limit_buckets():
    task5._rate_limit_buckets.clear()
    yield
    task5._rate_limit_buckets.clear()

def test_health_check(client):
    resp = client.get("/")
    assert resp.status_code == 200
    assert resp.get_json()["status"] == "ok"

def test_send_email_success(client, mocker):
    mock_response = mocker.Mock(status_code=202)
    mocker.patch.object(task5.SendGridAPIClient, "send", return_value=mock_response)

    resp = client.post(
        "/send_email",
        json={"recipient": "candidate@example.com", "candidate": "Jane Doe"},
        headers=AUTH_HEADERS,
    )

    assert resp.status_code == 200
    body = resp.get_json()
    assert body["status"] == "sent"
    assert body["recipient"] == "candidate@example.com"

def test_send_email_missing_recipient(client):
    resp = client.post(
        "/send_email",
        json={"candidate": "Jane Doe"},
        headers=AUTH_HEADERS,
    )
    assert resp.status_code == 400
    assert resp.get_json()["status"] == "failed"

def test_send_email_invalid_recipient_format(client):
    resp = client.post(
        "/send_email",
        json={"recipient": "not-an-email", "candidate": "Jane Doe"},
    )
```

```

        headers=AUTH_HEADERS,
    )
    assert resp.status_code == 400

def test_send_email_missing_candidate(client):
    resp = client.post(
        "/send_email",
        json={"recipient": "candidate@example.com"},
        headers=AUTH_HEADERS,
    )
    assert resp.status_code == 400
    assert resp.get_json()["status"] == "failed"

def test_send_email_sendgrid_exception_returns_502_and_hides_internals(client, mocker):
    mocker.patch.object(
        task5.SendGridAPIClient,
        "send",
        side_effect=Exception("raw internal sendgrid detail: bad api key XYZ"),
    )

    resp = client.post(
        "/send_email",
        json={"recipient": "candidate@example.com", "candidate": "Jane Doe"},
        headers=AUTH_HEADERS,
    )

    assert resp.status_code == 502
    body = resp.get_json()
    assert body["status"] == "failed"
    assert "raw internal sendgrid detail" not in body["error"]
    assert "XYZ" not in body["error"]

def test_send_email_requires_auth(client):
    resp = client.post(
        "/send_email",
        json={"recipient": "candidate@example.com", "candidate": "Jane Doe"},
    )
    assert resp.status_code == 401

def test_send_email_rejects_wrong_key(client):
    resp = client.post(
        "/send_email",
        json={"recipient": "candidate@example.com", "candidate": "Jane Doe"},
        headers={"X-API-Key": "wrong-key"},
    )
    assert resp.status_code == 401

def test_send_email_rate_limit(client, mocker):
    mock_response = mocker.Mock(status_code=202)
    mocker.patch.object(task5.SendGridAPIClient, "send", return_value=mock_response)

    original_max = task5.RATE_LIMIT_MAX_REQUESTS
    task5.RATE_LIMIT_MAX_REQUESTS = 2
    try:
        for _ in range(2):
            resp = client.post(
                "/send_email",
                json={"recipient": "candidate@example.com", "candidate": "Jane Doe"},
                headers=AUTH_HEADERS,
            )
            assert resp.status_code == 200

        resp = client.post(
            "/send_email",
            json={"recipient": "candidate@example.com", "candidate": "Jane Doe"},
            headers=AUTH_HEADERS,

```

```
)
    assert resp.status_code == 429
finally:
    task5.RATE_LIMIT_MAX_REQUESTS = original_max
```

## requirements.txt

*Python dependencies*

```
flask==3.0.3  
python-dotenv==1.0.1  
sendgrid==6.11.0  
pytest==8.2.0  
pytest-mock==3.14.0
```

## **.env.example**

*Environment variable template*

```
# Copy this file to .env and fill in real values. Never commit .env itself.

# --- task5.py (email notification service) ---

# Your SendGrid API key. Required for /send_email to actually send.
SENDGRID_API_KEY=your_sendgrid_api_key_here

# The "from" address used for outgoing notification emails.
FROM_EMAIL=notifications@example.com

# Shared-secret required in the X-API-Key header on every call to
# /send_email. Generate a long random value, e.g.:
# python -c "import secrets; print(secrets.token_urlsafe(32))"
APP_API_KEY=change_me_to_a_long_random_secret

# Set to true only for local development. Must be false (default) for any
# shared/production deployment -- see README.md.
FLASK_DEBUG=false

# Simple in-memory rate limiting for /send_email.
RATE_LIMIT_WINDOW_SECONDS=60
RATE_LIMIT_MAX_REQUESTS=10

# --- Interview_Lifecycle_Tracker/main.py ---

# Directory where events.jsonl is written. Defaults to ./Interview_Lifecycle_Tracker/data
TRACKER_DATA_DIR=./Interview_Lifecycle_Tracker/data

# Whether the tracker should call task5.py's /send_email on Completed/Incomplete.
TRACKER_NOTIFY_ENABLED=false
TRACKER_NOTIFY_URL=http://127.0.0.1:5000/send_email

# Must match APP_API_KEY above if TRACKER_NOTIFY_ENABLED=true.
TRACKER_NOTIFY_API_KEY=change_me_to_a_long_random_secret
```

## **.gitignore**

*Git ignore rules*

```
# Environment / secrets
.env

# Python
__pycache__/
*.pyc
.pytest_cache/
*.egg-info/
.venv/
venv/

# Tracker's local data
Interview_Lifecycle_Tracker/data/*.jsonl

# OS / editor cruft
.DS_Store
.vscode/
.idea/
```

# README.md

## Project documentation

### # Interview Notification Toolkit

Two small tools that work together as one interview-notification workflow:

- `task5.py` - a Flask service with a single `/send_email` endpoint that sends interview-status emails via SendGrid.
- `Interview_Lifecycle_Tracker/main.py` - a tkinter GUI for logging interview stage changes, with an optional (off by default) call into `task5.py` to trigger a real notification when a candidate is marked `Completed` or `Incomplete`.

### ## Setup

```
```bash
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
# edit .env: set SENDGRID_API_KEY and APP_API_KEY at minimum
```
```

### ## Running the email service

```
```bash
python task5.py
```
```

By default this starts the Flask dev server with debug mode `off`. To run with the interactive debugger for local development only:

```
```bash
FLASK_DEBUG=true python task5.py
```
```

Never set `FLASK_DEBUG=true` on a shared or production deployment - it enables Werkzeug's interactive debugger, which allows arbitrary code execution if the server is ever reachable beyond localhost. For anything beyond local iteration, run behind a proper WSGI server such as gunicorn or uwsgi, not Flask's built-in dev server.

### ### Endpoints

- `GET /` - health check, always returns `200 {"status": "ok"}`.
- `POST /send_email` - sends a notification email. Requires:
  - Header `X-API-Key: <APP_API_KEY>`
  - JSON body: `{"recipient": "someone@example.com", "candidate": "Jane Doe"}`

Responses now use accurate HTTP status codes instead of always returning `200`:

| Situation                                                        | Status |
|------------------------------------------------------------------|--------|
| ----- -----                                                      | -----  |
| Email sent successfully                                          | 200    |
| Missing/invalid <code>recipient</code> or <code>candidate</code> | 400    |
| Missing/invalid API key                                          | 401    |
| Rate limit exceeded                                              | 429    |
| Server misconfiguration (no keys set)                            | 500    |
| SendGrid call itself fails                                       | 502    |

Failure responses return a safe, generic message. The full exception detail is logged server-side (via the standard `logging` module) but never sent to the caller.

### ### Trust model

`/send_email` is a `thin internal service`, not a public-facing endpoint. It sends real email through a paid SendGrid account, so anyone who can reach

it and knows (or guesses) the API key can consume that account's sending quota. Treat it accordingly:

- Deploy it on an internal network / private subnet, not directly on the public internet.
- Set `APP\_API\_KEY` to a long random secret (see `.env.example`) and share it only with trusted internal callers (e.g. the tracker GUI, or another internal service).
- The built-in rate limiter (`RATE\_LIMIT\_MAX\_REQUESTS` per `RATE\_LIMIT\_WINDOW\_SECONDS`, per API key / IP) is in-memory and per-process. It bounds abuse from a single compromised or misbehaving caller on a single-worker deployment, but does **not** coordinate across multiple workers or replicas – back it with Redis (or similar) if you scale beyond one process.

## Running the tracker GUI

```
```bash
python Interview_Lifecycle_Tracker/main.py
```
```

Events are appended as structured JSON Lines to `Interview\_Lifecycle\_Tracker/data/events.jsonl` (configurable via `TRACKER\_DATA\_DIR`). Each line looks like:

```
```json
{"timestamp": "2026-07-18T10:15:00+00:00", "candidate": "Jane Doe", "stage": "Completed", "notes": ""}
```
```

If the log file can't be written (permissions, full disk, etc.), the GUI shows an error dialog instead of crashing.

### Wiring it to email notifications

By default the tracker does **not** call `task5.py`. To enable it:

```
```bash
export TRACKER_NOTIFY_ENABLED=true
export TRACKER_NOTIFY_URL=http://127.0.0.1:5000/send_email
export TRACKER_NOTIFY_API_KEY=<same value as APP_API_KEY>
```
```

When enabled, marking a candidate `Completed` or `Incomplete` (with a recipient email filled in) fires a background HTTP request to `/send\_email` on a daemon thread, so the GUI never blocks waiting on the network.

## Tests

```
```bash
pytest
```
```

`tests/test\_task5.py` covers, against a mocked `SendGridAPIClient`:

- the health-check route,
- a successful send,
- missing/invalid `recipient`,
- missing `candidate`,
- a SendGrid exception (asserts a `502` and that no raw exception text leaks into the response),
- missing/incorrect API key,
- and the rate limiter tripping after the configured request count.

## Dependencies

`requirements.txt` is intentionally minimal and UTF-8 encoded, listing only what this repo's code actually imports (`flask`, `python-dotenv`, `sendgrid`) plus test tooling (`pytest`, `pytest-mock`). `tkinter` is part of the Python standard library and isn't pip-installable – if it's missing on your system, install your OS's `python3-tk` package (or equivalent).